Implementación de la estructura completa de una base de datos

Red Social Estudiantil Pascualina

Integrantes:
Mariana Díaz Lastre
Jose David Celis Cepeda

Grupo:
7

Asignatura:
Bases de Datos I (G100)

Docente:
Oralia Cortes Grajales

Institución Universitaria Pascual Bravo
2026

1. Introducción

Este informe documenta la implementación física de la base de datos de la Red Social Estudiantil Pascualina en PostgreSQL. A partir del modelo lógico entregado en la fase anterior (diez tablas con sus claves y su diccionario de datos), se escribió el script DDL, se ejecutó en pgAdmin 4 y se verificó que las claves y restricciones funcionen como se diseñaron.

2. Selección del SGBD y justificación

2.1 Sistema elegido

Se eligió **PostgreSQL** como sistema gestor de bases de datos y **pgAdmin 4** como herramienta gráfica. Las razones principales son:

- **Cercanía al estándar ANSI SQL.** Soporta columnas `IDENTITY`, restricciones `CHECK` y las acciones referenciales `ON DELETE` y `ON UPDATE`, que son justamente lo que exige el modelo físico de esta tarea.
- **Código abierto y gratuito**, sin costos de licencia para un proyecto académico ni para una eventual puesta en marcha.
- **Capacidad de crecimiento.** Maneja tipos de datos avanzados (como JSON) y es extensible, lo que deja espacio para ampliar la red social sin cambiar de motor.
- **Adopción.** Según la Developer Survey 2024, citada en el libro de la unidad, PostgreSQL encabeza la lista de bases de datos usadas por los desarrolladores.
- **Herramienta gráfica.** pgAdmin 4 permite ejecutar el script, explorar las tablas y generar el diagrama entidad-relación sin depender solo de la línea de comandos.

2.2 Comparación con otras opciones

SGBD	Ventajas	Desventajas para este proyecto
PostgreSQL	Gratuito y de código abierto; amplio cumplimiento de ANSI SQL; `IDENTITY`, `CHECK` y acciones referenciales completas; tipos avanzados; pgAdmin 4.	Instalación y administración algo más técnicas que en una base embebida como SQLite.
MySQL	Gratuito y muy difundido en aplicaciones web (pila LAMP); MySQL Workbench; buen soporte transaccional.	Autoincremento con sintaxis propia (`AUTO_INCREMENT`); algunas restricciones estándar, como `CHECK`, solo se aplican en versiones recientes (desde la 8.0.16).
SQL Server	Herramienta muy completa (SSMS) e integración con el ecosistema Microsoft.	Licencia de pago en las ediciones completas (la Express es gratuita pero limitada); sintaxis propia `IDENTITY(1,1)`.
Oracle Database	Muy robusto, con capacidades empresariales para aplicaciones de alto rendimiento.	Costo de licencias y complejidad desproporcionados para un proyecto académico; el libro señala que el autoincremento se resuelve con secuencias y triggers.
SQLite	Ligero, sin servidor y portable; ideal para aplicaciones móviles o sistemas embebidos.	No es cliente-servidor: no se adapta a una red social con muchos usuarios simultáneos y ofrece menos herramientas de administración.

En conclusión, PostgreSQL ofrece el mejor equilibrio entre cumplimiento del estándar, costo, funciones para integridad de datos y herramientas de apoyo.

3. Del modelo lógico al modelo físico

El modelo lógico aprobado en la entrega anterior definió diez tablas, sus claves, restricciones y un diccionario de datos con el tipo y la longitud de cada atributo. La implementación física **no agregó ni eliminó tablas**: conservó esa estructura y tomó las decisiones propias de un SGBD real, que se resumen a continuación.

Elemento del modelo lógico	Cómo quedó en el modelo físico (PostgreSQL)
Diez entidades / tablas	Diez sentencias `CREATE TABLE` con los mismos nombres, en minúsculas, singular y separados por guion bajo.
Llaves primarias enteras autoincrementales	`INT GENERATED ALWAYS AS IDENTITY`: PostgreSQL genera el valor y no permite insertarlo a mano.
Llaves primarias compuestas (tablas intermedias)	`PRIMARY KEY (campo_1, campo_2)`, que evita relaciones repetidas.
Llaves foráneas	`FOREIGN KEY ... REFERENCES` con `ON DELETE` y `ON UPDATE`, decisión que solo se toma en el modelo físico (sección 5).
Atributos únicos (UK)	`UNIQUE` sobre `correo`, `nombre_interes` y `id_estudiante` en `perfil` (relación 1:1).
Tipos de datos del diccionario	Texto → `VARCHAR(n)` con la longitud del diccionario; entero → `INT`; "INT corto" → `SMALLINT`; DATETIME → `TIMESTAMP` (PostgreSQL no tiene DATETIME); DATE → `DATE`.
Reglas de dominio	`NOT NULL` en campos obligatorios; `DEFAULT CURRENT_TIMESTAMP` y `CURRENT_DATE` en fechas; `CHECK` para el semestre (1 a 12) y para que nadie se siga a sí mismo.
Nomenclatura	Reglas del libro: minúsculas con guion bajo, tablas en singular, claves con prefijo `id_` y restricciones con prefijos `pk_`, `fk_`, `uq_` y `ck_`.

4. Descripción de las tablas

4.1 Criterios generales para elegir los tipos de datos

- **VARCHAR(n) en lugar de TEXT.** En PostgreSQL ambos tienen el mismo rendimiento; la diferencia es el límite. Se usó `VARCHAR(n)` porque el modelo lógico ya definía una longitud para cada campo y así la base de datos rechaza textos que excedan lo previsto, lo que mantiene breves las biografías, descripciones y publicaciones.
- **INT en lugar de BIGINT.** `INT` (4 bytes) admite valores hasta unos 2.147 millones. Una red social universitaria no alcanzará esa cantidad de registros, y `BIGINT` ocuparía el doble de espacio en cada clave y en cada índice sin necesidad.
- **SMALLINT para el semestre.** Es un valor entre 1 y 12; `SMALLINT` ocupa la mitad que `INT`.
- **TIMESTAMP o DATE según la necesidad.** Publicaciones y eventos necesitan fecha y hora; la creación de un grupo solo requiere el día.

4.2 Detalle por tabla

4.2.1 estudiante

Guarda a cada persona de la red social; las demás tablas dependen de ella.

Campo	Tipo	Restricciones	Justificación
`id_estudiante`	INT (IDENTITY)	PK	Entero automático: único, corto y estable.
`nombre`	VARCHAR(50)	NOT NULL	50 caracteres alcanzan para nombres compuestos.
`apellido`	VARCHAR(50)	NOT NULL	Separado del nombre para buscar y ordenar.
`correo`	VARCHAR(100)	NOT NULL, UNIQUE	Correo institucional; único para evitar cuentas duplicadas.
`programa`	VARCHAR(80)	NOT NULL	Cubre los nombres más largos de programas.
`semestre`	SMALLINT	NOT NULL, CHECK (1 a 12)	Número pequeño y acotado.

Clave primaria: `id_estudiante`. Ningún dato personal sirve como identificador: el nombre se repite y el correo podría cambiar.

4.2.2 perfil

Presentación personal del estudiante, separada de los datos de la cuenta.

Campo	Tipo	Restricciones	Justificación
`id_perfil`	INT (IDENTITY)	PK	Mismo criterio que el resto de las claves.
`id_estudiante`	INT	NOT NULL, UNIQUE, FK	Dueño del perfil; el UNIQUE vuelve la relación 1:1.
`biografia`	VARCHAR(300)	Opcional	Presentación corta.
`foto`	VARCHAR(150)	Opcional	Se guarda la ruta o enlace, no la imagen.
`contacto`	VARCHAR(100)	Opcional	Teléfono o enlace externo.

Clave primaria: `id_perfil`. Aunque `id_estudiante` también es único, se conserva un identificador propio por consistencia con el modelo lógico aprobado.

4.2.3 publicacion

Mensajes que los estudiantes escriben en el muro.

Campo	Tipo	Restricciones	Justificación
`id_publicacion`	INT (IDENTITY)	PK	Mismo criterio que el resto de las claves.
`id_estudiante`	INT	NOT NULL, FK	Autor de la publicación.
`contenido`	VARCHAR(500)	NOT NULL	Más largo que una biografía, pero con tope definido.
`fecha_hora`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora juntas para ordenar el muro.

Clave primaria: `id_publicacion`. Un estudiante puede publicar el mismo texto varias veces, así que ningún dato identifica una publicación.

4.2.4 grupo

Comunidades que se forman en torno a una materia, un proyecto o un interés.

Campo	Tipo	Restricciones	Justificación
`id_grupo`	INT (IDENTITY)	PK	Mismo criterio que el resto de las claves.
`id_estudiante_creador`	INT	NOT NULL, FK	El sufijo `_creador` distingue este rol del de miembro.
`nombre_grupo`	VARCHAR(80)	NOT NULL	Título corto; sin UNIQUE, porque puede repetirse.
`descripcion`	VARCHAR(300)	Opcional	Descripción breve.
`fecha_creacion`	DATE	NOT NULL, DEFAULT CURRENT_DATE	Solo importa el día.

Clave primaria: `id_grupo`. El nombre puede repetirse o cambiar, por lo que no sirve como identificador.

4.2.5 evento

Reuniones de estudio, talleres o actividades sociales.

Campo	Tipo	Restricciones	Justificación
`id_evento`	INT (IDENTITY)	PK	Mismo criterio que el resto de las claves.
`id_estudiante_organizador`	INT	NOT NULL, FK	Responsable del evento; distinto del rol de asistente.
`id_grupo`	INT	FK, admite NULL	Grupo que lo promueve; opcional porque hay eventos independientes.
`nombre_evento`	VARCHAR(100)	NOT NULL	Títulos descriptivos.
`descripcion`	VARCHAR(300)	Opcional	Detalle de la actividad.
`fecha_hora`	TIMESTAMP	NOT NULL	Día y hora exactos para coordinar la asistencia.
`ubicacion`	VARCHAR(150)	NOT NULL	Espacio físico o enlace de reunión virtual.

Clave primaria: `id_evento`. Dos eventos pueden compartir nombre, lugar o fecha.

4.2.6 interes

Catálogo de aficiones para conectar estudiantes con gustos afines.

Campo	Tipo	Restricciones	Justificación
`id_interes`	INT (IDENTITY)	PK	Mismo criterio que el resto de las claves.
`nombre_interes`	VARCHAR(50)	NOT NULL, UNIQUE	Categorías cortas; único para no repetirlas en el catálogo.
`descripcion`	VARCHAR(200)	Opcional	Aclaración breve.

Clave primaria: `id_interes`, por el mismo criterio de identificador automático.

4.2.7 Tablas intermedias (relaciones N:M)

Resuelven las relaciones de muchos a muchos del modelo lógico. Todos sus campos son `INT` porque deben coincidir con el tipo de la clave primaria a la que apuntan, y todos son `NOT NULL`.

Tabla	Campos (todos INT, FK)	Clave primaria	Relación que resuelve
`estudiante_grupo`	`id_estudiante`, `id_grupo`	Compuesta: (`id_estudiante`, `id_grupo`)	Miembro entre estudiante y grupo.
`estudiante_evento`	`id_estudiante`, `id_evento`	Compuesta: (`id_estudiante`, `id_evento`)	Asistente entre estudiante y evento.
`estudiante_interes`	`id_estudiante`, `id_interes`	Compuesta: (`id_estudiante`, `id_interes`)	Intereses de cada estudiante.
`seguidor`	`id_seguidor`, `id_seguido`	Compuesta: (`id_seguidor`, `id_seguido`); además `CHECK (id_seguidor <> id_seguido)`	Relación reflexiva “sigue_a” entre estudiantes.

La clave primaria compuesta cumple dos funciones: identifica cada fila y evita que una misma relación se registre dos veces. Estas tablas no necesitan un identificador propio porque no tienen más atributos.

5. Descripción de las relaciones

El modelo físico tiene 13 claves foráneas. En la tabla, la “tabla hija” es la que contiene la clave foránea y la “tabla padre” es la que referencia.

Tabla hija → padre	Campo FK	Card.	ON DELETE	ON UPDATE	Justificación
perfil → estudiante	id_estudiante (UNIQUE)	1:1	CASCADE	CASCADE	Un perfil no tiene sentido sin su dueño.
publicacion → estudiante	id_estudiante	1:N	CASCADE	CASCADE	Un mensaje sin autor no tiene sentido en una red social.
grupo → estudiante	id_estudiante_creador	1:N	RESTRICT	CASCADE	Evita que desaparezca una comunidad con más miembros porque se fue su creador.
evento → estudiante	id_estudiante_organizador	1:N	RESTRICT	CASCADE	Mismo criterio: no perder un evento con asistentes al borrar a su organizador.
evento → grupo	id_grupo (opcional)	1:N	CASCADE	CASCADE	Si el grupo desaparece, sus eventos pierden contexto; los eventos sin grupo no se afectan.
estudiante_grupo → estudiante	id_estudiante	N:M	CASCADE	CASCADE	Si el estudiante se elimina, se elimina solo su membresía.
estudiante_grupo → grupo	id_grupo	N:M	CASCADE	CASCADE	Si el grupo se elimina, se eliminan sus membresías.
estudiante_evento → estudiante	id_estudiante	N:M	CASCADE	CASCADE	Se elimina solo la asistencia de ese estudiante.
estudiante_evento → evento	id_evento	N:M	CASCADE	CASCADE	Sin evento no hay asistencia que registrar.
estudiante_interes → estudiante	id_estudiante	N:M	CASCADE	CASCADE	Se eliminan solo los intereses de ese estudiante.
estudiante_interes → interes	id_interes	N:M	CASCADE	CASCADE	Si el interés sale del catálogo, se quitan sus asociaciones.
seguidor → estudiante	id_seguidor	N:M	CASCADE	CASCADE	Si el estudiante se va, desaparecen los seguimientos que hacía.
seguidor → estudiante	id_seguido	N:M	CASCADE	CASCADE	Si el estudiante se va, desaparecen los seguimientos que recibía.

5.1 Criterio aplicado

- **CASCADE en ON DELETE** cuando la fila hija solo tiene sentido mientras exista la fila padre: perfil, publicaciones y todas las tablas intermedias. Así no quedan datos huérfanos.
- **RESTRICT en ON DELETE** cuando la fila hija tiene valor para otras personas: un grupo o un evento no deben desaparecer en cadena porque su creador u organizador se retire. La base de datos obliga a reasignarlos o borrarlos de forma consciente.
- **CASCADE en ON UPDATE** en todas las relaciones, para que un cambio en el identificador del padre se propague a las tablas hijas y las relaciones no se rompan.

Nota técnica. Como las claves primarias se declararon `GENERATED ALWAYS`, PostgreSQL rechaza un `UPDATE` corriente sobre ellas (“la columna solo puede actualizarse a DEFAULT”). Por eso `ON UPDATE CASCADE` no actúa en la operación diaria; se dejó como configuración

preventiva, para que cualquier cambio administrativo del identificador (por ejemplo, durante una migración) se propague sin romper las relaciones.

6. Script DDL y su estructura

El modelo físico se entrega en tres scripts, ubicados en `tarea3/script`:

Archivo	Función
`red_social_pascualina_ddl.sql`	Script DDL: crea las diez tablas con sus claves y restricciones.
`red_social_pascualina_pruebas.sql`	Carga datos de ejemplo y comprueba que las restricciones rechacen los datos inválidos.
`red_social_pascualina_verificacion.sql`	Consultas que listan tablas, restricciones, acciones referenciales y filas por tabla.

El script DDL contiene únicamente diez sentencias `CREATE TABLE`, una por tabla. En cada una se declaran primero las columnas, con su tipo, `NOT NULL` y `DEFAULT`, y al final las restricciones con nombre (`pk_`, `uq_`, `ck_`, `fk_`), cada una en su propio bloque para que se lean y se localicen con facilidad.

Las tablas se crean en un orden que respeta las dependencias, porque una clave foránea solo puede apuntar a una tabla que ya existe:

Bloque	Tablas	Motivo del orden
1	`estudiante`, `interes`	No dependen de ninguna otra tabla.
2	`perfil`, `publicacion`, `grupo`	Dependen de `estudiante`.
3	`evento`	Depende de `estudiante` y de `grupo`.
4	`estudiante_grupo`, `estudiante_evento`, `estudiante_interes`, `seguidor`	Intermedias: dependen de dos tablas ya creadas.

Las reglas se aplicaron así: nombres en minúsculas con guion bajo, tablas en singular, claves primarias con prefijo `id_` (por ejemplo `id_estudiante`) y restricciones con prefijo según su tipo (`pk_estudiante`, `fk_perfil_estudiante`, `uq_estudiante_correo`, `ck_estudiante_semestre`). Las claves foráneas con roles distintos llevan un sufijo que aclara su función: `id_estudiante_creador`, `id_estudiante_organizador`, `id_seguidor` e `id_seguido`.

7. Implementación y verificación en pgAdmin 4

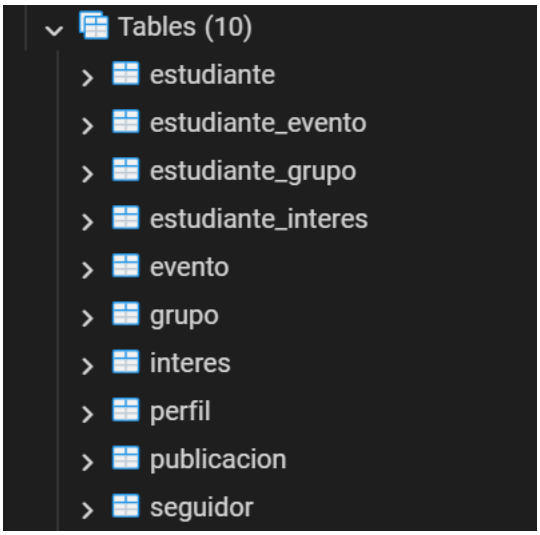


Figura 1. Tablas creadas en el esquema public.

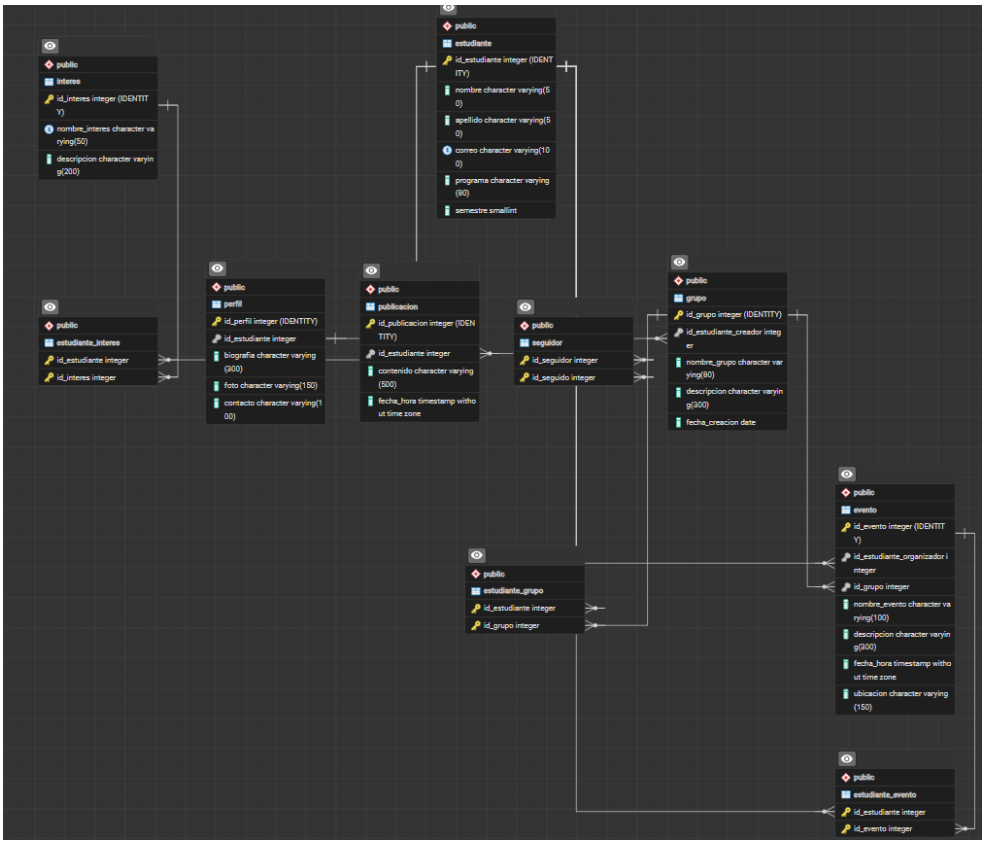


Figura 2. Diagrama entidad-relación generado por el SGBD.

7.1 Pruebas de las restricciones

El script de pruebas carga datos de ejemplo e intenta, dentro de bloques que capturan el error, operaciones que las restricciones deben impedir. Cada prueba imprime en la pestaña Messages de pgAdmin si la base de datos la rechazó y con qué mensaje:

Prueba	Qué se intenta	Restricción que actúa	Resultado esperado
1	Registrar un estudiante con un correo ya existente.	`uq_estudiante_correo`	Error: valor duplicado viola la restricción única.
2	Registrar un estudiante con semestre 15.	`ck_estudiante_semestre`	Error: la fila viola la restricción CHECK.
3	Crear un segundo perfil para el mismo estudiante.	`uq_perfil_estudiante`	Error: valor duplicado viola la restricción única.
4	Publicar con un `id_estudiante` inexistente (999).	`fk_publicacion_estudiante`	Error: la clave no está presente en la tabla `estudiante`.
5	Que un estudiante se siga a sí mismo.	`ck_seguidor_no_propio`	Error: la fila viola la restricción CHECK.
6	Registrar dos veces el mismo seguimiento.	`pk_seguidor`	Error: valor duplicado viola la clave primaria.
7	Insertar un `id_estudiante` escrito a mano.	`GENERATED ALWAYS`	Error: no se puede insertar un valor distinto de DEFAULT.
8	Borrar un estudiante que creó un grupo.	`fk_grupo_creador` (RESTRICT)	Error: la clave sigue referenciada desde `grupo`.
9	Borrar un estudiante que no creó grupos ni eventos.	`ON DELETE CASCADE`	Se borra, y sus filas en `estudiante_grupo` y `seguidor` desaparecen.

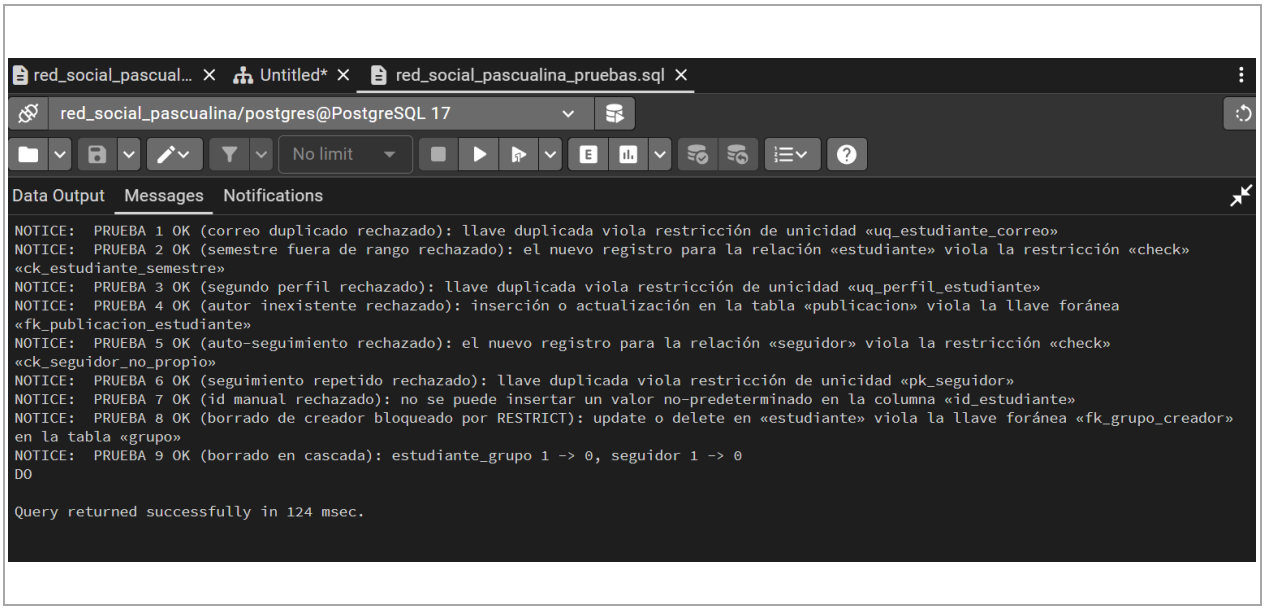
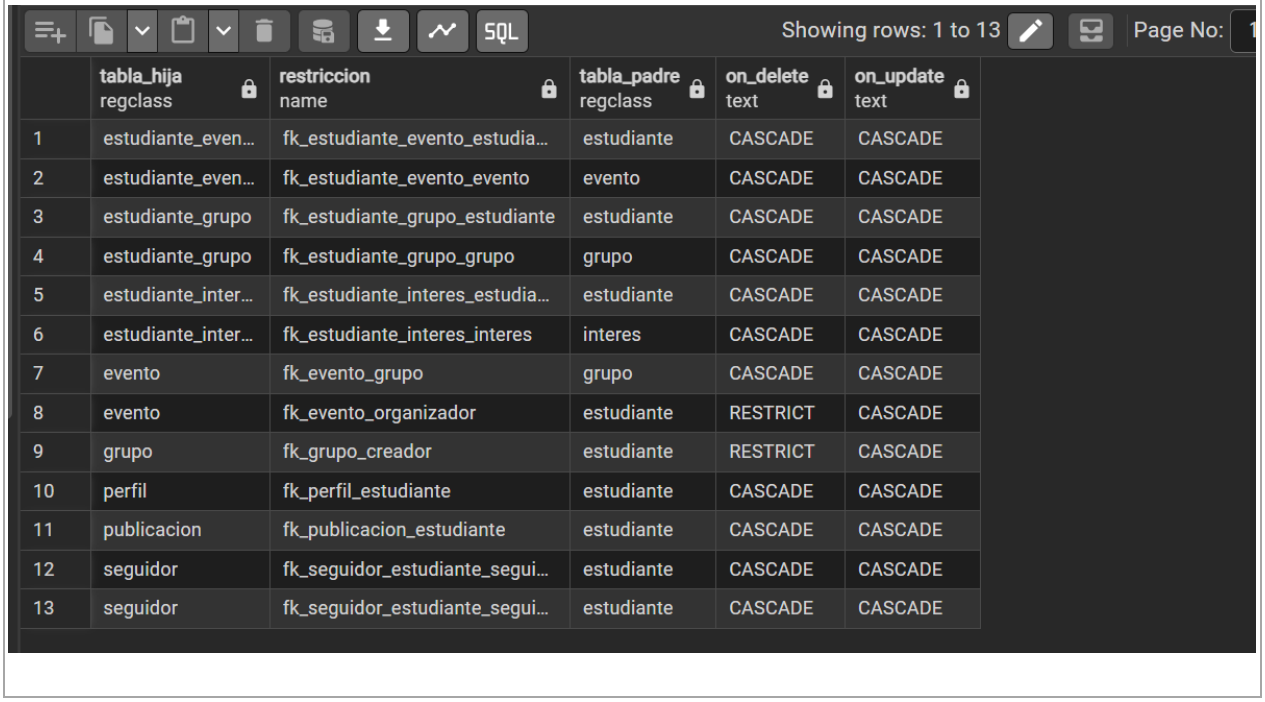


Figura 3. Resultado de las pruebas de integridad.

Además, el script de verificación lista todas las restricciones y las acciones referenciales de cada clave foránea directamente desde el catálogo del sistema, lo que permite confirmar que lo implementado coincide con lo descrito en la sección 5.



Showing rows: 1 to 13 Page No:

	tabla_hija regclass	restriccion name	tabla_padre regclass	on_delete text	on_update text
1	estudiante_even...	fk_estudiante_evento_estudia...	estudiante	CASCADE	CASCADE
2	estudiante_even...	fk_estudiante_evento_evento	evento	CASCADE	CASCADE
3	estudiante_grupo	fk_estudiante_grupo_estudiante	estudiante	CASCADE	CASCADE
4	estudiante_grupo	fk_estudiante_grupo_grupo	grupo	CASCADE	CASCADE
5	estudiante_inter...	fk_estudiante_interes_estudia...	estudiante	CASCADE	CASCADE
6	estudiante_inter...	fk_estudiante_interes_interes	interes	CASCADE	CASCADE
7	evento	fk_evento_grupo	grupo	CASCADE	CASCADE
8	evento	fk_evento_organizador	estudiante	RESTRICT	CASCADE
9	grupo	fk_grupo_creador	estudiante	RESTRICT	CASCADE
10	perfil	fk_perfil_estudiante	estudiante	CASCADE	CASCADE
11	publicacion	fk_publicacion_estudiante	estudiante	CASCADE	CASCADE
12	seguidor	fk_seguidor_estudiante_segui...	estudiante	CASCADE	CASCADE
13	seguidor	fk_seguidor_estudiante_segui...	estudiante	CASCADE	CASCADE

Figura 4. Acciones referenciales de las claves foráneas.

8. Conclusiones individuales y grupal

Mariana Díaz Lastre

Lo que más me dejó este trabajo es que escribir el DDL obliga a decidir cosas que en el dibujo del modelo se pasan por alto. Cada restricción, desde un campo obligatorio hasta una llave foránea, protege los datos de errores que nadie debería tener que revisar a mano. Lo más interesante fue pensar qué pasa cuando se borra un registro: tiene sentido que el perfil de un estudiante se vaya con él, pero no que un grupo con más gente desaparezca. Entendí que diseñar una base de datos también es cuestión de criterio, no solo de sintaxis.

Jose David Celis Cepeda

Al crear la base de datos con DDL entendí mejor cómo se organizan las tablas y cómo se relacionan entre sí. También aprendí que las claves y restricciones ayudan a mantener la información ordenada y evitar errores, como repetir un correo o relacionar una publicación con un estudiante que no existe.

este trabajo permitió comprender cómo se crea una base de datos y cómo se relacionan sus tablas mediante SQL.

PostgreSQL y pgAdmin 4 facilitaron la implementación de la red social.

Además, el uso de claves y restricciones ayuda a mantener la información organizada y reduce los errores al almacenar los datos. En conclusión, se logró establecer una estructura para almacenar y relacionar la información necesaria para el funcionamiento de la red social estudiantil.